

[← Previous](#)

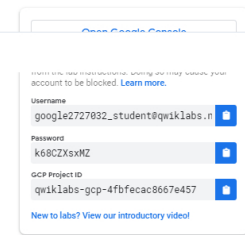
 This lab may incorporate AI tools to support your learning.

© 2023 Google

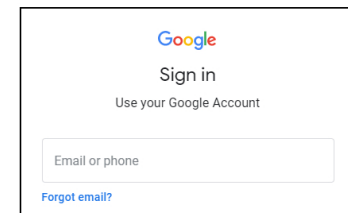
Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Google Cloud Console

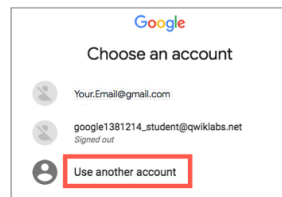
1. Click the **Start Lab** button. If you need to pay for the lab, a pop-up opens for you to select your payment method. On the left is a panel populated with the temporary credentials that you must use for this lab.



2. Copy the username, and then click **Open Google Console**. The lab spins up resources, and then opens another tab that shows the **Sign in** page.



If you see the **Choose an account** page, click **Use Another Account**.



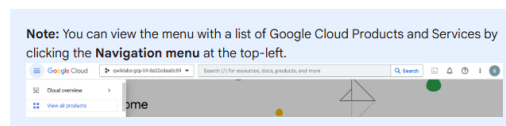
3. In the **Sign in** page, paste the username that you copied from the Connection Details panel. Then copy and paste the password.

Important: You must use the credentials from the Connection Details panel. Do not use your Qwiklabs credentials. If you have your own Google Cloud account, do not use it for this lab (avoids incurring charges).

4. Click through the subsequent pages:

- temporary account).
- Do not sign up for free trials.

After a few moments, the Cloud Console opens in this tab.



Task 1. Understand our app & architecture diagram

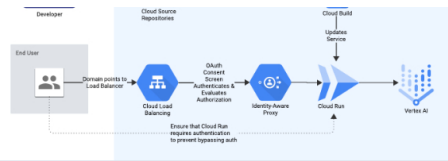
This app is written in Python using the popular web microframework [Flask](#). It uses a release called [Docker-Flask](#) to each implement the [Docker](#) front-end framework.

a nice or without needing to design it from scratch yourself.

Below is an architecture diagram of what you'll deploy. Consider the top half first, then we'll look at the bottom half.

You'd like to make it easy to maintain and improve your app, so you will use a deployment option that Cloud Run provides to deploy the app using a Cloud Build Trigger. This allows for developers to update the code base by testing locally and then merging code to a given branch of a repo, triggering Cloud Build to deploy the updated version of the service to Cloud Run.





The bottom half of the diagram shows how you can protect your app from unauthorized access using [Identity-Aware Proxy](#). Identity-Aware Proxy allows you to control access to a resource without needing to implement a sign-in screen, a forgot password flow, and other elements of a user management system. Instead you can grant access to users, Google Groups, or whole domains by assigning them the IAM role of "IAP-secured Web App User" using the Google Cloud Console.

As a whole, this pattern allows for quick deployment & maintenance of a simple web app designed to serve internal users or a small pool of external users.

Task 2. Push the code to a Github repository & review it

1. Open Cloud Shell by selecting the **Cloud Shell** icon at the top of the Google Cloud Console () and selecting **Continue**.
2. Download the app's code using a `gcloud storage cp` command with the

```
gcloud storage cp -r gs://pls-resource-bucket/tldr-  
everything .
```

3. If prompted to authorize Cloud Shell, select **Authorize**.
4. This folder of code isn't a git repository yet, so let's navigate into the folder and initialize the directory as a repo.

```
cd tldr-everything  
git init  
git checkout -b main  
git add .  
git config --global user.email "you@example.com"  
git config --global user.name "Qwiklabs Demo Account"  
git commit -m "initial commit"
```

5. Open a new browser tab, navigate to [GitHub.com](https://github.com) and sign in. If you don't have a Github account, you can create one for free. You'll return to this tab to create a repo shortly.

using the command line:

```
gh auth login
```

7. Select **GitHub.com** as the type of account to log into, then **HTTPS** for the protocol, confirm you will authenticate Git with GitHub credentials with a **Y**, and then select **Login with a web browser**.
8. Copy the one-time code provided to you by the `gh` command line tool. Then press **Enter** and click the link to go to <https://github.com/login/device>. Login. You may need to complete additional steps to login and confirm your account if you have multi-factor authentication enabled (as you should).
9. You will then be prompted to paste the code displayed in Cloud Shell after the `gh auth login` step above. Paste the code and proceed.
10. Select **Authorize GitHub**. You should see a message like "Congratulations, you're all set!" You can close this tab.
11. In the tab where you originally logged in to GitHub, select the **New** button () to create a new repo.

13. Select **Private**.

14. Then select **Create repository**.

15. In your new repository, use the copy button to copy the commands under the header **...or push an existing repository from the command line**. They should look like this, but be specific to your account:

```
git remote add origin https://github.com/YOUR-ACCOUNT-  
NAME/summarizer-app.git  
git branch -M main  
git push -u origin main
```

Now, if GitHub displays a "Repository already exists" error, you can create a

note: If Github displays a `permission denied (publickey)` error, generate an SSH key using Cloud Shell and add it to your Github account to grant access to Git. You can follow the steps outlined in the [Github documentation](#).

16. Return to Cloud Shell to paste this code copied from Github, and run it. After the code is pushed, you can close the Cloud Shell pane.

18. Select `app.py` to open our app's main file and scroll to the import statements.

```
import logging
import vertexai
from vertexai.generative_models import GenerativeModel,
GenerationConfig

from langchain.document_loaders import PyPDFLoader # to
load and parse PDFs
import markdown # to format LLM output for web display

import os # to remove temp PDF files
import uuid # to generate temporary PDF filenames

from flask import Flask, render_template, redirect,
url_for, session
from flask_bootstrap import Bootstrap5
from flask_wtf import FlaskForm, CSRFProtect
from flask_wtf.file import FileField, FileRequired,
FileAllowed
from wtforms.fields import SubmitField, TextAreaField
```

19. Notice a few key packages you are importing:

package	note
<code>from langchain.document_loaders</code> <code>import PyPDFLoader</code>	langchain gives you a few convenient Document loader classes to import and work with different sources of text .
<code>wtforms</code> and <code>flask_wtf</code>	WTForms lets you build forms in your Flask app using Python.
<code>markdown</code>	This package can help format text, like that from your LLM output, for web display.

20. Below the imports and a few configuration & initialization lines, notice the class definition for our upload form written in Python. See that it includes file-type validation from the `wtforms` and `flask_wtf` packages. This upload field will accept only files with `.pdf` extensions:

```
class UploadForm(FlaskForm):
    """A basic form to upload a file and take a text
    prompt."""
    pdf_file = FileField(
        validators=[FileRequired(),
        FileAllowed(['pdf'], 'Please select a PDF.')],
        label="Select a PDF".
```

```
        default="Summarize the PDF.")
    submit = SubmitField()
```

21. Below this class, we see the initialization of our Vertex AI client and the configuration of our model: For this use case, we are selecting a low temperature for more accuracy (rather than variation) in our model's response.

Working with `top_p` means that for each word the model generates, we are restricting its choice of words to the minimal set of words that will account for this portion of the probability distribution of possible next words. So a smaller `top_p` means choosing from a more restricted and predictable word each time.

```
vertexai.init(project=os.environ["PROJECT_ID"],
location="us-central1")

model = GenerativeModel("gemini-pro")

generation_config = GenerationConfig(
    temperature=0.3,
    top_p=0.6,
    candidate_count=1,
    max_output_tokens=4096,
)
```

22. At our default Flask route, the application displays the form by instantiating the `UploadForm()` class created above. When the user submits the form, the app checks if the validations have passed, then creates a temporary filename to save the PDF, load its text using langchain's `PyPDFLoader`, and combine the multiple pages of content into `combined_text`.

```
@app.route("/", methods=["GET", "POST"])
def index():
    """Route to display the input form and query the
    LLM."""
    form = UploadForm()
    if form.validate_on_submit():
        pdf_temp_filename = str(uuid.uuid4())
        form.pdf_file.data.save(pdf_temp_filename)
        loader = PyPDFLoader(pdf_temp_filename)
        pages = loader.load_and_split()
```



```
combined_text = "\n\n".join([p.page_content for p
in pages])
```

23. One way we could improve this app is to implement a [chain to handle longer \(or multiple\) documents](#), but for now, we will use the general guidance that 100 tokens correspond to about 60-80 words, giving us a conservative ratio of about 600 tokens per word (100 * 60 = 6000). We shall now create a function to

You can see here our prompt is very simple, opening with the input to our UI's 'instructions' text field, then specifying that the model should use information from the PDF to respond, then a label to identify the PDF text ("PDF:") and the PDF text itself.

```
word_count = len(combined_text.split())
# 18500 words times ~1.66 tokens per words should
keep us under Gemini's token limit:
# https://ai.google.dev/models/gemini#model-
variations
if word_count < 18500:
    prompt = f"Instructions: {form.text_input.data}
Use information from the PDF to
respond.\n\nPDF:\n{combined_text}"
    logging.debug(prompt)
    response = model.generate_content(prompt,
generation_config=generation_config)
    response_text = response.text.replace(" ", "
*")
else:
    response_text = "This text is too long for this
application's current configuration.\n\nPlease use a
shorter text."
```

With this, we parse the response using the `markdown` package to convert the model's response to HTML to display it on our `pdf_results` template:

```
markdown_response =
markdown.markdown(response_text)
session["markdown_response"] = markdown_response
logging.debug(f"Response: \n{markdown_response}")
os.remove(pdf_temp_filename)
return redirect(url_for("pdf_results"))
```

25. In addition to `app.py`, another key file is our `Dockerfile`, which builds on a basic Python container, installs our Python requirements, and then serves the app on the port 8080, which is the port Cloud Run sends requests to by default.

26. While we haven't gone over all the code in the app, the above highlights cover our key elements.

1. Navigate to Vertex AI and then click **Enable All Recommended APIs**.

Click *Check my progress* to verify the objective.



Enable Vertex AI APIs

Check my progress

Successfully enabled Vertex AI API

Task 4. Deploy a Cloud Run Service

1. Search for or navigate to Cloud Run in the Cloud Console and select "Service" under **Deploy Container**.

2. Select the radio button for **"Continuously deploy from a repository"**. This option

Cloud Run

Create service


Artifact Registry


Docker Hub

☐ Deploy one revision from an existing container image

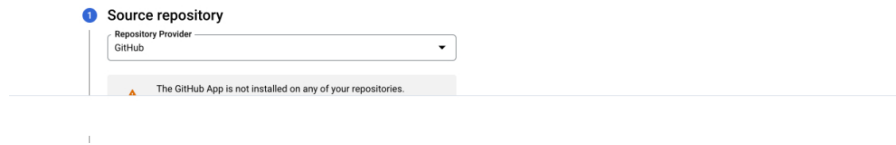
☒ Continuously deploy from a repository

SET UP WITH CLOUD BUILD

Required

3. Select the "GitHub" Repository Provider and the **Authenticate** link in the tool tip below it.

4. If prompted, select the **Install Google Cloud Build** button.



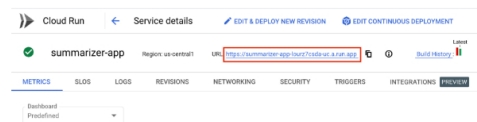
5. If prompted for a Google identity, select your Qwiklab student credentials to proceed.
6. On the GitHub "Install Google Cloud Build" page, select your GitHub username. You may choose to install Google Cloud Build on all repositories. If in doubt, select **Only select repositories** and the repository you created for this lab (`summarizer-app`). Select **Install**.
7. Back in the Google Cloud Console, in the "Set up with Cloud Build" panel, select the "summarizer-app" Repository. Confirm the terms by **selecting the checkbox**, then select **Next**.
8. In the Build Configuration settings, keep the Branch set to `^main$`, so only pushes or merges to your `main` branch will trigger deployments. In a real deployment setting, you could set up separate Cloud Run services, perhaps in different environment projects, to deploy from `dev`, `test`, and `main` branches.
9. For Build Type select "Dockerfile" and leave the Source location to the default of "/Dockerfile" because the Dockerfile for this deployment is at the root level of this app's repo structure. Click "Save".
10. Back in the Cloud Run deployment settings, ensure the Region is set to `us-`

Note: Cloud Run displays only the region where the lab is provisioned.

11. Under Authentication, select **Allow unauthenticated invocations**. We are only selecting this option to test our service, and we will set this to "Require authentication" after configuring Identity-Aware Proxy.
12. Under "Billing", select **Request-based** (Charged only when processing requests. CPU is limited outside of requests.)
13. Under "Service autoscaling", set "Minimum number of instances" to 0. You could scale this up if you did not want users to have to wait for instances to spin up when no one has used the app recently.
14. Expand the "**Container(s), Volumes, Networking, Security**" section by selecting its name, and select the **Containers** tab.
15. Within the Containers tab, scroll to "Revision autoscaling" and set the **Maximum number of instances** to 1.
16. Scroll back up to see tabs for "Settings", "Variables & Secrets" and "Volume Mounts" within the Containers tab, and select the **Variables & Secrets** tab.

Name of "PROJECT_ID" and the value of `qwiklabs-gcp-03-2ea653f2a2a7`.

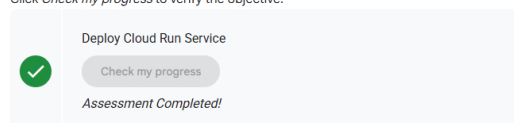
18. You can leave other parameters set to their default values and select "Create".
19. Navigate to Cloud Build in the Google Cloud Console, then click the "Triggers" navigation item on the left to see the Trigger that has been created for this service.
20. You can also select the "History" nav item on the left to see the current Build. Select its "Build" hash value to see the steps in progress.
21. When the build has completed, navigate back to Cloud Run and select the service. Select the service's URL to access it:



22. Try uploading a PDF (if you need one as an example, here is a report on [2023 Google Carbon Removal Research Awards](#)) and viewing the summary you receive.

a question about the content of the PDF (for example for the PDF linked above, "In what areas of study is Google encouraging more research related to carbon removal?").

Click *Check my progress* to verify the objective.



Task 5. Put the app behind a Load Balancer

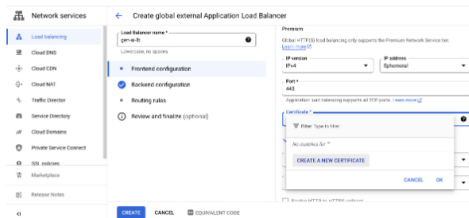
To protect your Cloud Run service with Identity-Aware Proxy, first put it behind a load balancer.

- From the IP Addresses page, select **Reserve external static IP address**, name the IP something like "summarizer-app-lb-ip", set the Type to "Global" and select "Reserve".
- Copy and paste the IP Address assigned to you in a notes document to use a few steps later.
- Navigate to "Load balancing" by searching for it at the top of the Cloud Console and selecting the option associated with "Network services".
- Select **Create Load Balancer**

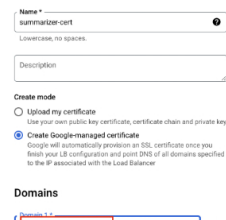


- Select **Application Load Balancer (HTTP/HTTPS)** and **Next**.
- Select **Public facing (external)** and **Next**.
- Select **Best for global workloads** and **Next**.

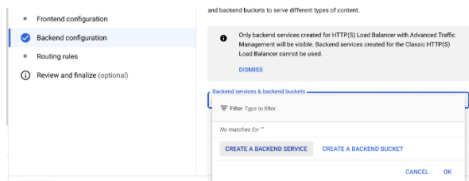
- Select **Configure**.
- Name the load balancer (ex. "summarizer-app-lb"), then on the right-hand side, name the frontend (ex. "summarizer-app-frontend").
- Select a Protocol of **HTTPS (includes HTTP/2 and HTTP/3)**.
- Under "IP address" select the named **global IP address** you created.
- Under Certificate, select "Create a new certificate."



In a real deployment setting, you would enter your domain (ex. summarizer.com) or a subdomain (ex. summarizer.google.com) and then [create A \(or AAAA if using IPv6\) DNS records to forward traffic to the load balancer's static IP address](#). For this demo, we will use a handy website, [nip.io](#), which routes traffic to an IP provided to it as a subdomain. So copy the static IP address you created earlier add ".nip.io" to the end of it so that it reads like "NNN.NNN.NNN.NNN.nip.io" with your IP address digits replacing the Ns. Paste that into the "Domain 1" box of your certificate creation model. When we complete our setup, you will be able to access your app at "https://NNN.NNN.NNN.NNN.nip.io" (again with your IP address digits replacing the Ns). Select "Create."



- Back in the load balancer's configuration, select "Backend configuration" then the "Backend services & backend buckets" dropdown and then "Create a backend service."



17. Name our backend (ex. "summ-backend"). For a backend type, select "Serverless network endpoint group", then under the "New backend" header, select the "Serverless network endpoint groups" dropdown and then "Create serverless network endpoint group".

Name *

Lowercase, no spaces

Description

Backend type

Serverless network endpoint group

Protocol

HTTPS

Named port

http

Timeout

30 seconds

Backends

New backend

Serverless network endpoint groups *

Filter Type to filter

No matches for "

CREATE SERVERLESS NETWORK ENDPOINT GROUP

ADD X BACKEND

Cloud CDN

Enable Cloud CDN

Cloud CDN is a paid product that accelerates web content and applications. [Learn more about features and pricing.](#)

CREATE CANCEL

18. In the "Create a backend service" panel, select "Cloud Run" as your backend type, then select our deployed service and select "Create".

Create Serverless network endpoint group

Name *

summ-backend

Lowercase, no spaces

Region *

us-central1 (Iowa)

Serverless network endpoint group type

Cloud Run

Select service

Service *

summ-backend (us-central1)

Traffic tag

Use URL mask

Cloud Functions

App Engine

CREATE CANCEL

19. Back in the "Create a backend service" panel, disable the "Enable Cloud CDN" option (which isn't compatible with Identity-Aware Proxy).

20. Select "Create" to finish creating the backend.

21. Select "Create" one more time to create your load balancer.

22. On the Load balancing services page, we can see a spinning icon as the load balancer is created. Once it is created, we can click on the load balancer's name to see more details. In the Frontend section, we can select the certificate's name (under "Certificate") to see our SSL certificate's status. This should move from "PROVISIONING" to "ACTIVE" within about 10-20 minutes (though in some cases it could take longer). You can proceed a little further with the lab while this provisions, and we will tell you when we need to pause to wait for this to receive an "ACTIVE" status.

summary-cert

Certificate type

MANAGED

Status

ACTIVE

Status description

The Google-managed SSL certificate is obtained from the Certificate Authority

Domains

summary-gen-41.pory

ACTIVE

DNS Hostnames

summary

Expires

Feb 11, 2024, 3:55:17 PM

Serial number

10000000000000000000000000000000

Certificate issuer

GTS CA 104

In use by

gts-ai-16-target-proxy

Certificate chain

GTS Root R1 - Jan 27, 2020, 7:00:42 PM

Click [Check my progress](#) to verify the objective.

Configure Load Balancer

Check my progress

Assessment Completed!

Task 6. Enable Identity-Aware Proxy

You can now configure Identity-Aware Proxy to grant access to the application.

1. Search for Identity-Aware Proxy (or "IAP") in the Console to navigate to it, then select "Enable API". Once the API is enabled, select "Go to Identity-Aware Proxy".

the app and to give them a chance to agree to that or not. Select "Configure Consent Screen".

Identity-Aware Proxy

APPLICATIONS SSH AND TCP RESOURCES CONNECTORS

Identity-Aware Proxy (IAP) lets you manage who has access to services hosted on App Engine, Compute Engine, or an HTTPS Load Balancer. [Learn more](#)

To get started with IAP, add an [App Engine app](#), a [Compute Engine instance](#) or configure an [HTTPS Load Balancer](#).

CONNECT NEW APPLICATION [Premium](#)

Before you can use IAP, you need to configure your OAuth consent screen.

CONFIGURE CONSENT SCREEN

3. On the "OAuth Overview" page, click on **GET STARTED**, then under "App Information" enter your app with a name (ex. "Summarizer App")
4. Provide a user support email address (you can use your Qwiklabs-generated student ID email).

Select "Next".

App Information

Audience

☐ Internal

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

NEXT

Contact Information

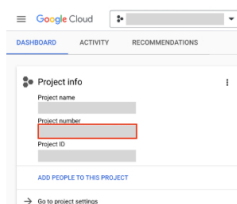
Finish

6. Add your Qwiklabs student email or your own email address as the "Contact Information" email address. Click "Save and continue". You can also click "Next".
7. Accept the terms and condition for "Google API Services: User Data Policy," then

click on **CREATE**.

9. On the **OAuth Overview** page, click on **Create Auth Client**, under Application type select **Web Application**.
 10. Under **Authorized redirect URIs** click on **+ ADD URL** and enter URL "https://NNN.NNN.NNN.NNN.nip.io" then click **Create**.
 11. On the "OAuth consent screen" page, navigate to **Audience**, under Publishing Status, select "Publish app" and then click "Confirm". This will allow you to add any Google identity to which you grant the "IAP-secured Web App User" role to access the app, rather than additionally needing to add them as a test user on the OAuth consent screen.
 12. Navigate back to the "Load Balancing" dashboard, select your load balancer, and then the Certificate name. If this is not yet ACTIVE, we will need to wait until it reaches ACTIVE status. Take a break and refresh occasionally.
 13. When the certificate is **ACTIVE**, navigate back to Identity-Aware Proxy by searching "IAP" at the top of the Console.
 14. Toggle on IAP protection of our backend service. The backend service may show a status of Error before you enable IAP, but enabling it should complete its service. You will be prompted to select a service account, which
- and select **TURN ON**.

15. Return to Cloud Run, select your service, and then the **Security** tab. Set the Authentication setting to **Require authentication** and select "Save." This is important to prevent your service from being accessible at its Cloud Run link directly, which would bypass the IAP protection. This change may take a few minutes to become active.

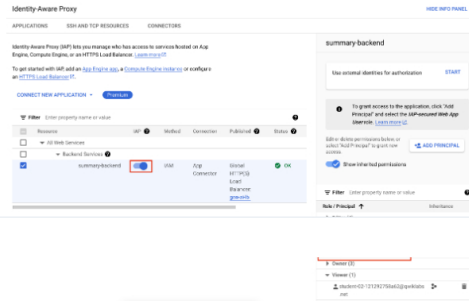


16. We need to grant the IAP service account a role within IAM. You will need your Project Number (a number that is different from your Project ID). You can find it by navigating to the Cloud Console's upper-left menu and selecting **Cloud Overview > Dashboard**. You'll see your project number in the upper left. Copy

NUMBER}@gcp-sa-iap.iam.gserviceaccount.com

17. Navigate to IAM, select **+ Grant Access**, and grant that service account the IAM Role of **Cloud Run Invoker**.

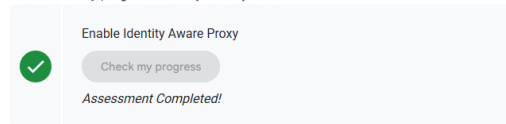
18. You can now grant access to individual users (betty@example.com), [Google Groups](#) (marketing@example.com), or entire domains (example.com) by granting them the "IAP-secured Web App User" IAM role. This role can be granted on the project overall, or on a single selected Backend Service within IAP. To use this fine-grained control, navigate back to the IAP dashboard, and select our backend service. Then select **+Add Principal** in the panel on the right.



19. Add a Google Identity of yours as a user with the "IAP-secured Web App User" role.

20. You may see an "Error: Forbidden" message for about the first 5 minutes, but after that users with the "IAP-secured Web App User" role on the Project or IAP backend service should be able to access the app via the domain we listed on our Load Balancer certificate (<https://NNN.NNN.NNN.NNN.nip.io> with your load balancer's IP address replacing the Ns). Make sure to use **https** and not simply **http** at the start of the URL.

Click *Check my progress* to verify the objective.



Congratulations

You have now completed the lab! In this lab, you deployed a Cloud Run app that can make calls to a Vertex AI Language Model. You also protected your app from unauthorized users using Identity-Aware Proxy.

Next steps

- Check out the [Generative AI on Vertex AI documentation](#).
- Learn more about Generative AI on the [Google Cloud Tech YouTube channel](#).

Google Cloud Training & Certification

...helps you make the most of Google Cloud technologies. [Our classes](#) include technical skills and best practices to help you get up to speed quickly and continue your learning journey. We offer fundamental to advanced level training, with on-demand, live, and virtual options to suit your busy schedule. [Certifications](#) help you validate and prove your skill and expertise in Google Cloud technologies.

Manual Last Updated March 21, 2025

Lab Last Tested March 21, 2025

Copyright 2023 Google LLC All rights reserved. Google and the Google logo are trademarks of Google LLC. All other company and product names may be trademarks of the respective companies with which they are associated.